A bootstrap ensemble of `HEFTNet` models

Plan for `heft_train_ensemble.ipynb` and `dihiggs/ensemble.py`

Revised plan: bootstrap of the training rows. For approval before any code is written

October 30, 2026 (supersedes the first version of the same date)

Abstract

The first Bayesian neural network run (`docs/bnn/bnn_run1.pdf`) gave credible bands about 17 times narrower than the model's actual error, partly because a single HMC chain explores a single valley of the posterior. This plan describes a complementary, frequentist approach: train many copies of `HEFTNet`, each on a *bootstrap* resample of the training rows and with its own random seed, and use the spread of their predictions as an estimate of the standard error of the trained network. A second ensemble, which varies only the seed, separates the part of that spread due to the optimisation (which valley) from the part due to the finite training sample. The first version of this plan resampled whole spectra; following our discussion, it now resamples the training data themselves, as the pipeline sees them. Which resampling is "right" is an instance of a classic and unresolved problem of statistical inference, the choice of the *reference ensemble*, discussed in section 3. The work is a new notebook, `code/heft_train_ensemble.ipynb`, and a small new module, `dihiggs/ensemble.py`. Nothing will be written until you approve; the decisions needed are collected in section 11.

The proposal in brief

1. **Two ensembles** of $M = 20$ members each (table 1): `'seeds'` (the training rows as they are, different seeds) and `'bootstrap'` (the training rows resampled with replacement, different seeds).
2. **The bootstrap embeds our sample in the population it was drawn from,** as seen by the pipeline: the training rows are regarded as independent draws from a (discrete, complicated) distribution P, and each bootstrap set is a draw of the same size from the empirical distribution of those rows.
3. **The split of `heft_train.ipynb` is kept exactly:** the same training, validation and quick-test rows. Only the training rows are resampled; the validation rows are the same for every member.
4. **Each member is trained exactly as in `heft_train.ipynb`** and saved as an ordinary run folder, so `heft_test.ipynb` and `heft_export.ipynb` work with any member.
5. **An `Ensemble` class** with the same methods as `hmc.BNN`.
6. **The test,** on the 1,000 holdout spectra: is the ensemble spread, added to the POWHEG noise, the size of the actual differences from POWHEG?
7. **The reference-ensemble problem** (section 3): the answer depends on the population into which the sample is embedded, and there is no unique choice. The plan states its choice

explicitly.

Contents

1 Why an ensemble **2**

2 What the ensemble estimates **3**
2.1 The frequentist view . . . 3
2.2 Separating the two sources . . . 3
2.3 How many members? . . . 4

3 The reference-ensemble problem **4**
3.1 The problem . . . 4
3.2 How it shows up in the bootstrap . . . 5

4 Resampling the training rows **6**

5 Training the members **7**

6 The module `dihiggs/ensemble.py` **7**

7 The notebook `heft_train_ensemble.ipynb` **8**

8 The test: is the spread the size of the error? **8**

9 Links with the BNN **9**

10 Not in this plan **9**

11 Decisions for you **9**

1 Why an ensemble

The BNN run showed three reasons why a single chain's band is too narrow (`bnn_run1.pdf`, §6): incomplete exploration of the valley it is in; other valleys, in which equally good networks give different predictions where the data constrain them weakly; and the model's own systematic error, which no spread of parameters within one model describes. An ensemble of independently trained networks addresses the second directly. Each training run, started from different random weights and fed batches in a different order, ends in a different valley. Large studies have found that such "deep ensembles" reproduce the predictions of long, full-data HMC better than most other approximations, precisely because they visit several valleys [20, 19, 18]. Resampling the training data as well, by the bootstrap [12, 13], adds the variation that any estimator has because its training set is one finite sample.

2 What the ensemble estimates

2.1 The frequentist view

Think of the whole training procedure as an *estimator*: it takes a training set D and a random seed s and returns a function $\hat{f}(\boldsymbol{x}; D, s)$. Had the training set been a different draw from the same population, or had the seed been different, the result would have been different. So $\hat{f}(\boldsymbol{x})$ at a fixed point $\boldsymbol{x}$ is a random variable, with a mean and a variance. Its standard deviation is the *standard error* of the trained network at $\boldsymbol{x}$, the frequentist counterpart of the width of a Bayesian band.

The bootstrap principle

We have only one training set, so the distribution of $\hat{f}$ over training sets cannot be observed directly. The bootstrap [12, 13] replaces the unknown distribution P from which the N training rows were drawn by their *empirical* distribution $\hat{P}$, which puts probability $1/N$ on each row. A bootstrap training set D^* is a sample of N rows drawn from $\hat{P}$, i.e. drawn from the training rows *with replacement*. On average it contains $1-(1-1/N)^N \approx 1-e^{-1} \approx 63.2\%$ of the distinct rows, some of them several times. The spread of $\hat{f}(\boldsymbol{x}; D^*_m, s_m)$ over $m = 1, \ldots, M$ estimates the spread of $\hat{f}$ over training sets drawn from P.

Your thought experiment

Suppose the colleague had generated a hundred times more data, and the pipeline were run on each of 100 disjoint pieces of it. The spread of the 100 trained networks is the quantity of interest: the sampling variability of the procedure at the size of data we actually have. Each piece is a draw from P, and the bootstrap mimics such draws by drawing from $\hat{P}$. The quality of the mimicry rests on $\hat{P}$ being close to P, which, with 274,152 training rows, is a reasonable expectation for the smooth functionals that a trained network represents.

2.2 Separating the two sources

The law of total variance splits the variance of $\hat{f}$ at $\boldsymbol{x}$ into an optimisation part and a data part:

$$\underbrace{\mathrm{Var}_{D,s}\left[\hat{f}\right]}_{\text{total}} = \underbrace{\mathrm{E}_D\Big[\mathrm{Var}_s\left[\hat{f} \mid D\right]\Big]}_{\text{optimisation: which valley}} + \underbrace{\mathrm{Var}_D\Big[\mathrm{E}_s\left[\hat{f} \mid D\right]\Big]}_{\text{data: which sample}}. \tag{1}$$

The two ensembles of table 1 estimate the first term and the total. The data term is their difference, assuming that the optimisation spread is about the same for a bootstrap set as for the original set.

`MODE`	training rows of member m	seed	estimates	run folder
`'seeds'`	the 274,152 training rows	`SEED` $+m$	optimisation term	`runs/ENS_seeds`
`'bootstrap'`	274,152 rows drawn from them	`SEED` $+m$	total	`runs/ENS_bootstrap`

Table 1: The two ensembles. Member m uses the same seed in both, so the two members start from the same initial weights; only the training rows differ.

What neither ensemble captures: the shared bias

All members have the same architecture, the same loss and the same training schedule. If they all miss the plateau just above threshold in the same way, they agree with each other, and the spread stays small however many members there are. The bootstrap standard error is an honest error bar on the *estimator* $\hat{f}$, not on the true cross section, unless the bias $\mathrm{E}[\hat{f}] - \sigma_{\mathrm{true}}$ is small compared with it. Near threshold it may not be. The test of section 8 is designed to show whether it is.

2.3 How many members?

The standard deviation of M draws is itself uncertain, by a relative amount of about $1/\sqrt{2(M-1)}$:

M	10	20	50	100
relative uncertainty of the standard deviation	24%	16%	10%	7%

I propose $M = 20$ per ensemble, enough to tell a factor of two in width, which is the level of the questions we are asking. Members can be added later without redoing the others.

3 The reference-ensemble problem

3.1 The problem

Every frequentist statement, a standard error, a confidence level, a p-value, is a statement about an *ensemble*: the imagined infinite collection of repetitions of the experiment of which the one we performed is regarded as a member. The data do not determine that ensemble. A single sample can be embedded in many populations, each consistent with how it was obtained, and each giving a different answer. This is the statistical form of the *reference class problem*, as old as the frequency theory of probability itself: Venn [1] noted that an individual belongs to many classes, in which the frequency of an attribute differs, and Reichenbach [2] proposed, without a general prescription for finding it, to take the narrowest class for which reliable statistics exist. Hájek [3] argues that every interpretation of probability, the Bayesian included, faces a version of the problem.

Within statistics proper, Fisher [4] insisted that the reference set must contain no *recognisable subset*: no subset, identifiable from the data, in which the frequencies differ. Cox's famous example [5] makes the point. A measurement is made with one of two instruments, one precise and one imprecise, chosen by the toss of a coin. The ensemble of all repetitions, in which either instrument may be used, gives a valid confidence interval for the unconditional coverage; but once we know which instrument was used, the relevant ensemble is surely the repetitions with *that* instrument, and the unconditional interval is too wide for one and too narrow for the other. Buehler [6] and Robinson [7] turned Fisher's remark into a criterion and showed that standard confidence intervals can have *relevant subsets* in which their coverage is systematically different. The conditionality principle, made formal by Birnbaum [8] (see also [9]), says one should condition on such ancillary information; carried to its conclusion, together with sufficiency, it leads to the likelihood principle, under which the ensemble disappears altogether, which is precisely what the frequentist cannot accept. The problem is therefore not a technical gap but a fault line between schools of inference; Cox and Hinkley [10] and, for physicists, Cousins [11] give balanced accounts.

An analogy from statistical mechanics

The same system can be described by the microcanonical, canonical or grand canonical ensemble, depending on what is held fixed: the energy, the temperature, the particle number. The ensembles agree on some quantities and not on others, notably fluctuations. The choice is not dictated by the system but by the question asked and by what is regarded as fixed in the imagined repetitions. Reference ensembles in statistics behave in the same way.

3.2 How it shows up in the bootstrap

The bootstrap does not escape the problem; it makes it concrete. The resampling scheme *is* the choice of reference ensemble. For regression, the classic distinction is between resampling the *pairs* $(\boldsymbol{x}_i, t_i)$, which treats the inputs as random, and resampling the *residuals* at fixed inputs, which treats them as fixed [14, 15, 16]. Freedman showed that both are valid, under their own assumptions, and that they estimate different variances when the model is wrong or the noise heteroscedastic; Davison and Hinkley's guiding rule is that the resampling scheme should mimic the way the data were generated [16], which presupposes an answer to the question of which features of that way are to be held fixed. For our data at least three embeddings are defensible:

resample	imagined repetition	regarded as fixed
training rows (this plan)	new independent draws of rows $(\boldsymbol{x}, t)$ from P	the population P of rows, as the pipeline sees them
whole spectra	a new set of HEFT points θ, each with a new POWHEG run	the distribution from which θ is drawn
POWHEG noise only	new events at the *same* HEFT points (a residual or parametric bootstrap, using the known event weights w_j)	the HEFT points and the bins

Each answers a well-posed question, and the questions differ in what is held fixed. None is uniquely correct, and the data cannot choose between them. The row bootstrap chosen here treats the training data as the pipeline does: a large collection of rows from one distribution, with no further structure. It is the most direct expression of your thought experiment, in which the pipeline is simply re-run on more data of the same kind.

The working stance

1. **State the ensemble.** Every number this ensemble produces is a standard error with respect to repetitions in which the training rows are independent draws from the population P of rows. The report will say so.
2. **Calibrate empirically.** Whatever the reference ensemble, the holdout spectra provide a test that does not depend on it: whether the spread, combined with the POWHEG noise, matches the actual differences from POWHEG (section 8). This is the frequentist's own way out: coverage, checked on data not used for training.
3. **Treat the other embeddings as sensitivity checks,** if the result invites them. If, for example, the bootstrap spread proves too small near threshold, the noise bootstrap would tell whether the POWHEG statistics alone account for more.

And the Bayesian?

The Bayesian conditions on the data actually observed and needs no ensemble of repetitions; Cox's two instruments raise no difficulty. But the problem returns in another form: the choice of the model and the prior, and the likelihood itself, embody assumptions about how the data arose. Our BNN, for example, treats the 274,152 rows as conditionally independent given the parameters, which is also a statement about the population. The posterior predictive checks of Gelman, Meng and Stern [17] face the analogous choice explicitly: which features of the data are held fixed in the replications.

4 Resampling the training rows

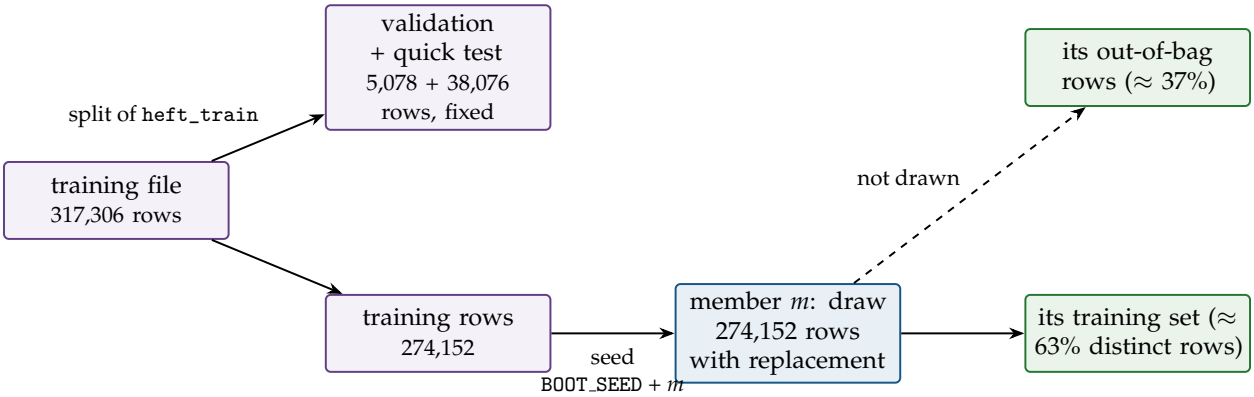


Figure 1: How the training set of one bootstrap member is formed. The split into training, validation and quick-test rows is that of `heft_train.ipynb`, the same for all members of both ensembles.

- **The split.** `heft_train.ipynb` takes the first 86.4% of the rows of the (already shuffled) training file for training, the next 1.6% for validation and the last 12% as the quick test. The ensemble notebook uses exactly the same split, so a `'seeds'` member is the `heft_train` model with a different seed, and only the 274,152 training rows are resampled. The validation rows are therefore never copies of training rows, and the validation loss, which chooses the saved parameters, is unaffected by the resampling.
- **The draw.** Member m draws 274,152 indices uniformly, with replacement, from the training rows, with `np.random.RandomState(BOOT_SEED + m)`, whose sequence is frozen across numpy versions. The drawn rows are in random order, so no further shuffling is needed.
- **Bookkeeping.** Each bootstrap member's folder gets `bootstrap_counts.npz`: for each training row, how many times it was drawn (compressed, about 100 kB). Together with `BOOT_SEED` this makes every member reproducible and identifies its *out-of-bag* rows, those it never saw, which give a free extra test of that member.
- **Size.** A bootstrap set has as many rows as the original, so the training schedule needs no change.

5 Training the members

Each member is trained with the code of `heft_train.ipynb`, unchanged except for the training rows: L1 loss, Adam, batch size 2,048, 200,000 iterations in four learning-rate steps from 2×10^{-3}, parameters saved at the best validation loss. Before member m is built, `torch.manual_seed(SEED + m)` and `np.random.seed(SEED + m)` fix its initial weights and its batch order.

Run folders. Member m of an ensemble is saved in `runs/ENS_<mode>/member_01/`, `member_02/`, ..., each an ordinary run folder (`dihiggs_config.yaml`, `dihiggs_params.pth`, `dihiggs_losses.csv`), plus `bootstrap_counts.npz` for bootstrap members. The configuration records `MODE`, the member number, both seeds and the training file's SHA-256. A summary file `runs/ENS_<mode>/ensemble.csv` has one row per member: seeds, number of distinct rows, best validation loss, out-of-bag loss, training time.

Interruption and parallel running. Members are trained one after another, and a member whose parameter file and summary row already exist is skipped, so an interrupted loop resumes where it stopped. A switch `MEMBERS` (e.g. `range(0, 10)`) chooses which members a notebook trains, so that two or more copies of the notebook can train disjoint sets of members at the same time on the 5090. `HEFTNet` is small; several trainings fit on one GPU at once. Given the unstable connection to the 5090, running the notebook headless (`jupyter nbconvert --execute` inside `tmux`) is recommended.

The real-time loss monitor. `heft_train.ipynb` opens a window that plots the losses during training. In a loop over 20 members that would open 20 windows, so the ensemble notebook writes the loss files without starting the window; progress is printed instead.

Cost. About 42 minutes per member on a CPU; on the 5090 I expect a few minutes. The notebook times the first member and prints an estimate for the rest. The `N_ITERATIONS` switch allows shorter members for a first trial.

6 The module `dihiggs/ensemble.py`

A class `Ensemble` with the same interface as `hmc.BNN`, so that analysis code can use either:

```
from dihiggs import ensemble
from dihiggs import heftnet as NN

ens = ensemble.Ensemble(NN.HEFTNet, 'runs/ENS_bootstrap')  # every member_*/
mu  = ens.mean(x)                  # ensemble mean, one value per row of x
sd  = ens.std(x)                   # spread of the members (the standard error)
lo, hi = ens.interval(x, cl=0.68)  # central 68% of the members
A   = ens.apply(lambda m, x: m.coeffs(x), xs)   # a_i(m_hh) for every member
```

It reads `member_*/dihiggs_params.pth` by default (any file pattern can be given), and also provides `summary()`, the contents of `ensemble.csv`. Like `hmc.py`, it imports nothing from `dihiggs`, so it can move to `drwho`. `Ensemble` is written as a general "ensemble of parameter files"; when both modules move to `drwho`, `hmc.BNN` can become a subclass of it. `hmc.py` is not changed now.

7 The notebook `heft_train_ensemble.ipynb`

In the style of `heft_train.ipynb` (header, Colab cell with `USE_DRIVE`, imports, device, data with its SHA-256), then:

1. **Introduction** (markdown): the estimator view, the bootstrap, the decomposition (1), and a short statement of the reference ensemble used.
2. **Switches:** `MODE = 'bootstrap'` or `'seeds'`; `N_MEMBERS = 20`; `MEMBERS = range(N_MEMBERS)`; `SEED = 42`; `BOOT_SEED = 1000`; `N_ITERATIONS = 200_000`. `DIRNAME` = `ENS_<mode>`.
3. **The split** of `heft_train.ipynb`.
4. **One member's data:** a function returning the training rows of member m and its counts.
5. **One member's training:** a function that builds the configuration, model, optimiser, scheduler and monitor, trains, computes the out-of-bag loss and appends the summary row; the training loop is that of `heft_train.ipynb`.
6. **The loop** over `MEMBERS`, skipping finished members.
7. **Results on the holdout spectra** (section 8): the ensemble mean against POWHEG; spectra with ensemble bands; the spread against m_{hh}; the pull test; and, when both ensembles exist, the decomposition (1) against m_{hh}, and the comparison with the BNN run if there is one.

8 The test: is the spread the size of the error?

On the 1,000 holdout spectra, at every bin k with POWHEG value t_k, POWHEG noise σ_k (as in `heft_test.ipynb`), ensemble mean $\bar{f}_k$ and ensemble standard deviation s_k, form the pull

$$z_k = \frac{t_k - \bar{f}_k}{\sqrt{\sigma_k^2 + s_k^2}}. \tag{2}$$

If the ensemble spread describes the model's error, and the error and the POWHEG noise are independent, z_k has mean 0 and standard deviation 1, and $|z_k| < 2$ for about 95% of bins. If the spread is too small, the standard deviation of z is larger than 1. The same pull, with s_k the posterior standard deviation, is computed for the BNN, so all are compared on one scale. Note that this test does not depend on the choice of reference ensemble (section 3): it asks only whether the stated uncertainty matches the observed errors on spectra not used in training.

quantity	reported
accuracy of the ensemble mean	median χ^2/N; fractions above 2 and 5; mean χ^2 per bin at 255–555 GeV and above; against the single members and the L1 model
width	s_k/σ_k: median, distribution, against m_{hh}
calibration	standard deviation of z; fraction $\lvert z\rvert < 2$; both against m_{hh}
decomposition	optimisation and data terms of (1) against m_{hh}
out-of-bag (bootstrap only)	each member's loss on the rows it did not see, against its validation loss

What counts as success.

1. The ensemble mean is at least as accurate as a single L1 model, as averaging over members usually makes it.
2. The spread is *informative*: larger where the model is known to be weak (near threshold) than elsewhere.
3. The standard deviation of z is close to 1 above about 555 GeV. Near threshold I expect it to exceed 1, because of the shared bias; how much it exceeds 1 measures that bias, which is itself a useful number.

9 Links with the BNN

- **Starting points for several HMC chains.** The members of the `'seeds'` ensemble lie in different valleys. Starting four BNN chains from four of them makes $\hat{R}$ across chains a real test for several valleys. This needs a small option in `heft_train_bnn.ipynb`, `START_FROM`, a list of parameter files, one per chain, to be added after this plan.
- **A common yardstick.** The pull (2) puts the BNN band, the two ensemble spreads and the POWHEG noise on one scale.

10 Not in this plan

- No change to `heft_train.ipynb`, `heft_test.ipynb`, `hmc.py` or the data files.
- The spectrum and noise bootstraps of section 3; the noise bootstrap is a natural sensitivity check later, if the results call for one.
- The `START_FROM` option of the BNN notebook, and the tempered likelihood, are separate steps.
- Bootstrap confidence intervals more refined than the spread (percentile, BCa) are left out: with $M = 20$ only the standard deviation is well determined.
- Nothing is committed to git.

11 Decisions for you

Decisions before I proceed

1. **Two ensembles,** `'seeds'` and `'bootstrap'` (rows), $M = 20$ each. Agreed, or start with 10?
2. **The split of** `heft_train.ipynb`, with only the 274,152 training rows resampled. Agreed?
3. **The training recipe** of `heft_train.ipynb`, unchanged (L1, 200,000 iterations). Agreed?
4. **No real-time monitor window** during ensemble training; printed progress instead. Agreed?
5. `Ensemble` **as a new module,** `dihiggs/ensemble.py`, leaving `hmc.py` unchanged for now. Agreed?

6. **Folder names:** `runs/ENS_seeds/member_01/`, ... Acceptable?
7. **This document** replaces the first version at `docs/ensemble/ensemble_plan.pdf`. Agreed?

References

[1] J. Venn, *The Logic of Chance*, Macmillan (1866).

[2] H. Reichenbach, *The Theory of Probability*, University of California Press (1949).

[3] A. Hájek, "The reference class problem is your problem too", Synthese **156** (2007) 563.

[4] R. A. Fisher, *Statistical Methods and Scientific Inference*, Oliver and Boyd (1956).

[5] D. R. Cox, "Some problems connected with statistical inference", Ann. Math. Statist. **29** (1958) 357.

[6] R. J. Buehler, "Some validity criteria for statistical inferences", Ann. Math. Statist. **30** (1959) 845.

[7] G. K. Robinson, "Some counterexamples to the theory of confidence intervals", Biometrika **62** (1975) 155.

[8] A. Birnbaum, "On the foundations of statistical inference", J. Amer. Statist. Assoc. **57** (1962) 269.

[9] J. O. Berger and R. L. Wolpert, *The Likelihood Principle*, 2nd ed., IMS Lecture Notes–Monograph Series **6** (1988).

[10] D. R. Cox and D. V. Hinkley, *Theoretical Statistics*, Chapman & Hall (1974).

[11] R. D. Cousins, "Why isn't every physicist a Bayesian?", Am. J. Phys. **63** (1995) 398.

[12] B. Efron, "Bootstrap methods: another look at the jackknife", Ann. Statist. **7** (1979) 1.

[13] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall/CRC (1993).

[14] D. A. Freedman, "Bootstrapping regression models", Ann. Statist. **9** (1981) 1218.

[15] C. F. J. Wu, "Jackknife, bootstrap and other resampling methods in regression analysis", Ann. Statist. **14** (1986) 1261.

[16] A. C. Davison and D. V. Hinkley, *Bootstrap Methods and their Application*, Cambridge University Press (1997).

[17] A. Gelman, X.-L. Meng and H. Stern, "Posterior predictive assessment of model fitness via realized discrepancies", Statistica Sinica **6** (1996) 733.

[18] B. Lakshminarayanan, A. Pritzel and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles", NeurIPS 2017, arXiv:1612.01474.

[19] A. G. Wilson and P. Izmailov, "Bayesian deep learning and a probabilistic perspective of generalization", NeurIPS 2020, arXiv:2002.08791.

[20] P. Izmailov, S. Vikram, M. D. Hoffman and A. G. Wilson, "What are Bayesian neural network posteriors really like?", ICML 2021, arXiv:2104.14421.